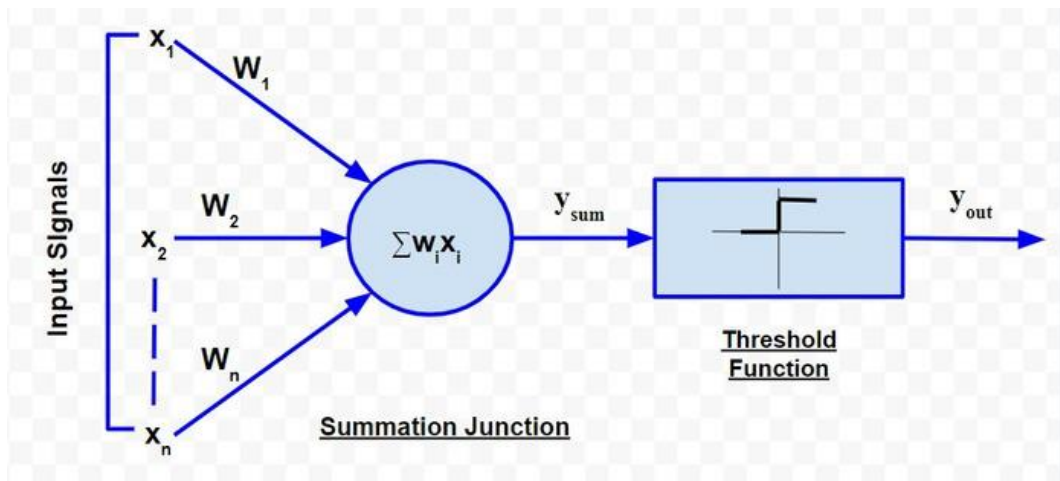


Module 1

Q. Model of McCulloch Pitts Neuron

=>



Introduction

1. The McCulloch Pitts neuron is the first mathematical model of an artificial neuron proposed in 1943 by Warren McCulloch and Walter Pitts.
2. The model was designed to imitate the working of biological neurons using simple logical operations.
3. The neuron receives multiple inputs and produces a single output based on a threshold decision.
4. The model is also called a **Threshold Logic Unit (TLU)** because it uses a threshold function to decide the output.
5. The McCulloch Pitts neuron is considered the foundation of modern neural networks and deep learning.

Components of McCulloch Pitts Neuron

1. The model consists of input signals represented as $(x_1, x_2, x_3, \dots, x_n)$.
2. Each input has an associated weight represented as $(w_1, w_2, w_3, \dots, w_n)$.
3. The weights represent the importance or strength of each input signal.
4. A summation unit calculates the weighted sum of all input signals.
5. The summation operation is mathematically written as $(y_{sum} = \sum (w_i x_i))$.
6. A threshold function compares the weighted sum with a predefined threshold value.
7. The output neuron produces a binary result depending on whether the threshold is crossed or not.
8. The output is usually represented as (y_{out}) .

Working of the McCulloch Pitts Neuron

1. The neuron first receives input signals from external sources or other neurons.

2. Each input signal is multiplied by its corresponding weight value.
3. The neuron calculates the total weighted sum using the summation junction.
4. The calculated sum is passed to the threshold activation function.
5. The neuron produces output 1 if the weighted sum is greater than or equal to the threshold.
6. The neuron produces output 0 if the weighted sum is less than the threshold.
7. The output is therefore always binary in nature.

Mathematical Model

1. The mathematical representation of the neuron is

$$y = f(\sum w_i x_i).$$

2. The activation function used is a step or threshold function.
3. The output rule can be written as:

- Output = 1 if $\sum w_i x_i \geq \theta$
- Output = 0 if $\sum w_i x_i < \theta$

4. The symbol (θ) represents the threshold value of the neuron.

Example

1. A McCulloch Pitts neuron can be used to implement logical AND operation.
2. Consider two inputs (x_1) and (x_2) with weights equal to 1.
3. Let the threshold value be 2.
4. The output becomes 1 only when both inputs are 1.
5. The neuron behaves exactly like an AND gate in digital electronics.
6. The model can also implement OR logic by changing weights and threshold values.

Characteristics of McCulloch Pitts Neuron

1. The neuron works only with binary inputs and binary outputs.
2. The neuron does not learn automatically because weights are fixed manually.
3. The activation function is a non-linear step function.
4. The model is simple and computationally efficient.
5. The neuron can solve linearly separable problems.

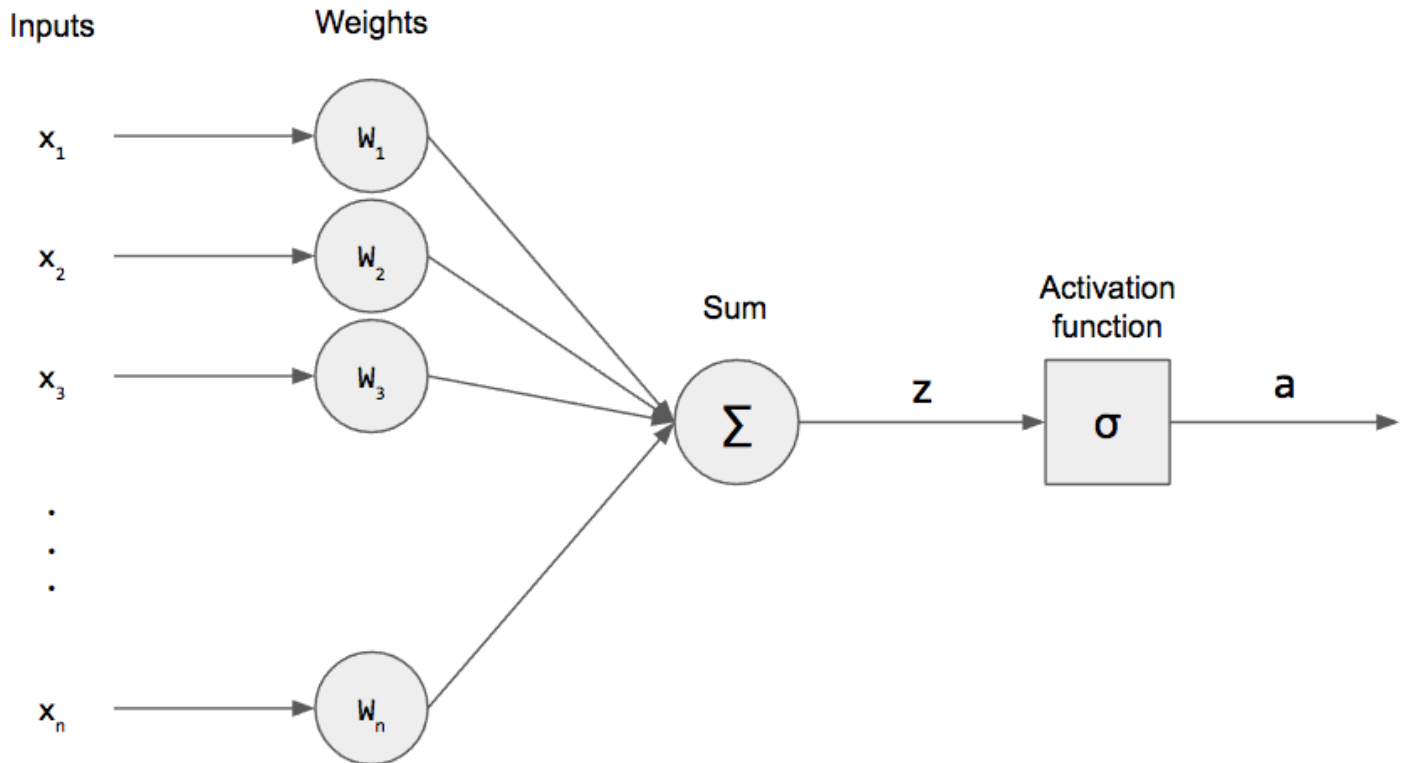
Applications

1. The McCulloch Pitts neuron is used for understanding the basic concept of neural networks.
2. The model is useful for implementing simple logical circuits.
3. The neuron forms the conceptual base for perceptron and deep learning models.
4. The model is widely used in educational demonstrations of artificial intelligence concepts.

Q. Implementation of Logical AND Gate using McCulloch Pitts (MP) Neuron

=>

refer this link : <https://www.youtube.com/watch?v=-rxxleYcZzU>



Introduction

1. A logical AND gate produces an output of 1 only when all inputs are equal to 1.
2. The McCulloch Pitts neuron can implement logical operations because it works using threshold logic.
3. The AND function is linearly separable, so it can be implemented using a single MP neuron.
4. The neuron uses weighted inputs and a threshold value to generate the correct output.

Components Required

1. The implementation requires two binary input signals represented as (x_1) and (x_2) .
2. Each input is assigned a weight value represented as (w_1) and (w_2) .
3. A summation unit calculates the weighted sum of inputs.
4. A threshold activation function decides whether the neuron should fire or not.
5. The output is represented as (y) .

Truth Table of AND Gate

1. The AND gate has four possible input combinations for two inputs.

2. The output is 1 only when both inputs are 1.

Input (x1) | Input (x2) | Output (y)

0 | 0 | 0

0 | 1 | 0

1 | 0 | 0

1 | 1 | 1

Mathematical Implementation

1. Let the weights be $w_1 = 1$ and $w_2 = 1$.
2. Let the threshold value be $\theta = 2$.
3. The weighted sum is calculated as $y_{sum} = w_1x_1 + w_2x_2$.
4. The activation function compares the sum with the threshold value.
5. The neuron produces output 1 if $y_{sum} \geq 2$.
6. The neuron produces output 0 if $y_{sum} < 2$.

Working Explanation

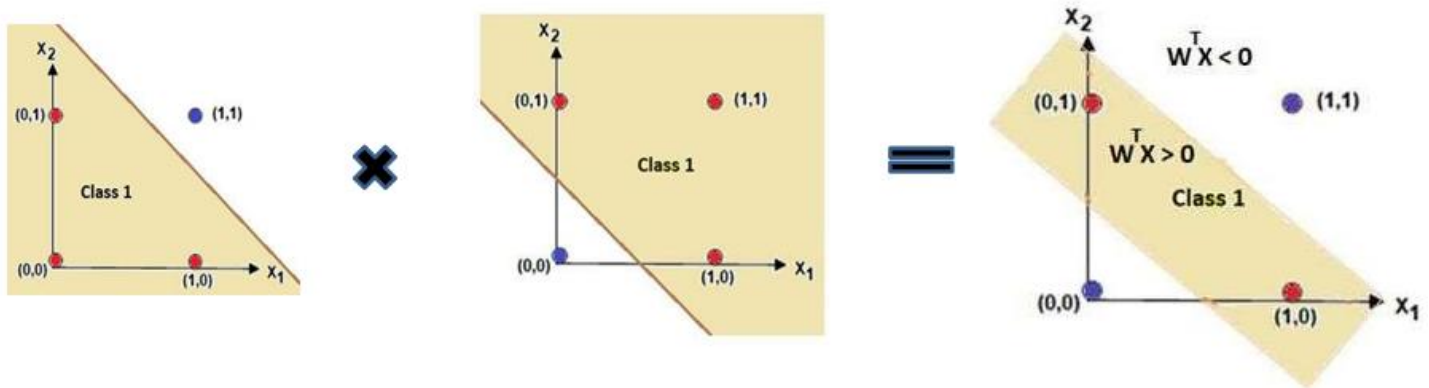
1. When both inputs are 0, the weighted sum becomes 0, which is less than the threshold, so the output is 0.
2. When one input is 1 and the other is 0, the weighted sum becomes 1, which is still less than the threshold, so the output is 0.
3. When both inputs are 1, the weighted sum becomes 2, which is equal to the threshold, so the output becomes 1.
4. The neuron therefore behaves exactly like an AND gate.

Example Calculation

1. Consider inputs ($x_1 = 1$) and ($x_2 = 1$).
2. The weighted sum becomes ($1 \times 1 + 1 \times 1 = 2$).
3. The threshold condition is satisfied because the sum equals 2.
4. The output of the neuron becomes 1.

Q. Why McCulloch Pitts (MP) Neuron is Considered a Linear Classifier?

=>



Truth Table - NAND		
X1	X2	Output Class
0	0	1
0	1	1
1	0	1
1	1	0

Truth Table - OR		
X1	X2	Output Class
0	0	0
0	1	1
1	0	1
1	1	1

Truth Table - XOR		
X1	X2	Output Class
0	0	0
0	1	1
1	0	1
1	1	0

Basic Idea

1. A linear classifier is a model that separates data using a straight line.
2. The McCulloch Pitts neuron separates inputs into two classes based on a mathematical equation.
3. The equation used by the MP neuron is a linear equation.
4. Therefore, the boundary created by the MP neuron is always a straight line.

Working Reason

1. The MP neuron multiplies each input with a weight value.
2. The neuron adds all the weighted inputs together to get a total sum.
3. The neuron compares this sum with a threshold value.
4. The neuron gives output 1 if the sum is greater than the threshold.
5. The neuron gives output 0 if the sum is less than the threshold.
6. This comparison creates a straight line boundary between two output classes.

Mathematical Reason

1. The neuron uses the formula ($y = w_1x_1 + w_2x_2 + \dots + w_nx_n$).
2. This formula is a linear equation because there are no powers or complex terms.
3. A linear equation always produces a straight line or plane.

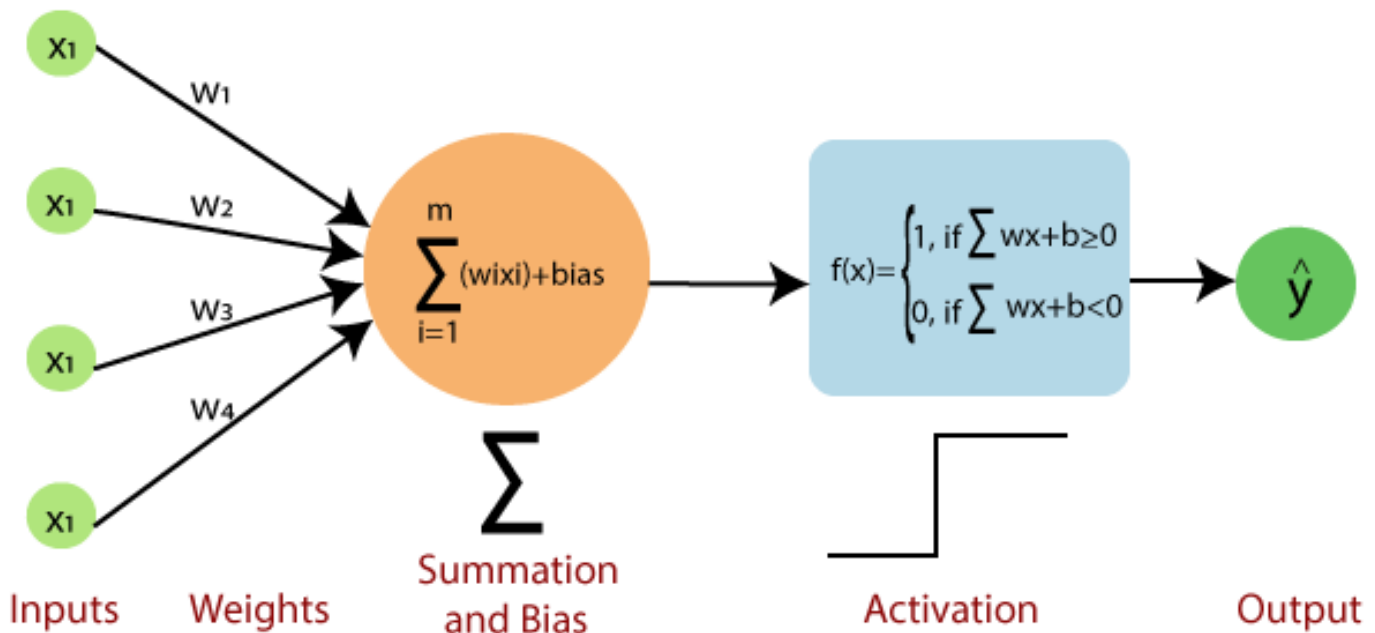
4. Hence, the MP neuron performs linear classification.

Example

1. The AND gate problem can be separated using a straight line between output 0 and output 1 points.
2. The MP neuron successfully solves the AND gate because it is linearly separable.
3. The XOR problem cannot be separated using a single straight line.
4. The MP neuron fails for XOR because it is only a linear classifier.

Q. Architecture of the Perceptron

=>



Introduction

1. The perceptron is one of the simplest types of artificial neural network models.
2. The perceptron was proposed by Frank Rosenblatt in 1957.
3. The perceptron is mainly used for binary classification problems.
4. The perceptron works similar to the McCulloch Pitts neuron but it can learn weights automatically.
5. The perceptron consists of input units, weights, bias, summation unit, and activation function.

Components of Perceptron Architecture

1. Input Layer

1. The perceptron receives input signals represented as $(x_1, x_2, x_3, \dots, x_n)$.
2. Each input represents a feature of the data.
3. The input layer does not perform any computation.
4. The input layer only passes the values to the next stage.
5. Example: In email spam detection, inputs can be number of links, number of keywords, and message length.

2. Weights

1. Each input is associated with a weight value represented as $(w_1, w_2, w_3, \dots, w_n)$.
2. The weight indicates the importance of each input feature.
3. Higher weight means the input has more influence on the output.
4. The perceptron learns these weights during training.

3. Bias

1. The perceptron includes a bias term represented as (b) .
2. The bias helps to shift the decision boundary.
3. The bias allows the model to classify data even when all inputs are zero.
4. The bias improves the flexibility of the model.

4. Summation Unit

1. The summation unit calculates the weighted sum of inputs and bias.
2. The mathematical formula is $(z = \text{sum of } w_i x_i + b)$.
3. The summation combines all information into a single value.

5. Activation Function

1. The activation function converts the summation value into output.
2. The perceptron usually uses a step or sign activation function.
3. The output becomes 1 if the value is above a threshold.
4. The output becomes 0 or -1 if the value is below the threshold.

6. Output Layer

1. The perceptron produces a single output value.
2. The output represents the predicted class.
3. The perceptron is therefore called a single-layer neural network.

Working of Perceptron

1. The perceptron receives input values from the dataset.
2. Each input is multiplied by its corresponding weight.
3. The weighted inputs are added along with the bias.
4. The activation function is applied to the result.
5. The perceptron produces the final output class.

Example

1. A perceptron can classify whether a student will pass or fail based on study hours and attendance.
2. Inputs can be study hours and attendance percentage.
3. The perceptron calculates a weighted sum and predicts pass or fail.

Characteristics

1. The perceptron can learn from training data using a learning algorithm.
2. The perceptron works well for linearly separable problems.
3. The perceptron is simple and computationally efficient.
4. The perceptron is the foundation of modern neural networks.

Q. Derivation of Perceptron Weight Update Rule

=> refer this link : <https://youtu.be/8rhjp2xcD-o?si=iyrltJr6CKuPF337>

Introduction

1. The perceptron learning rule is used to update the weights of the perceptron during training.
2. The main goal of the perceptron is to reduce the classification error.
3. The perceptron adjusts its weights whenever the predicted output is wrong.
4. The learning rule helps the perceptron learn the correct decision boundary.

Step 1: Output of Perceptron

1. The perceptron calculates the weighted sum of inputs and bias.
2. The mathematical equation is ($y = f(w \cdot x + b)$).
3. The symbol ($w \cdot x$) represents the dot product of weight and input vectors.
4. The function (f) represents the activation function.

Step 2: Error Calculation

1. The perceptron compares the predicted output with the target output.
2. The error is calculated as ($e = t - y$).
3. The symbol (t) represents the target or actual output.
4. The symbol (y) represents the predicted output.
5. The error indicates how much correction is required.

Step 3: Idea of Weight Correction

1. If the output is correct, then no weight update is needed.
2. If the output is wrong, then weights must be adjusted.
3. The weight change should be proportional to the error.
4. The weight change should also depend on the input value.

Step 4: Weight Update Formula

1. The change in weight is represented as Δw .
2. The perceptron learning rule states $\Delta w = \eta \cdot e \cdot x$.
3. The symbol η represents the learning rate.
4. The learning rate controls how fast the model learns.
5. The new weight becomes $w_{new} = w_{old} + \Delta w$.

Step 5: Final Perceptron Update Rule

1. The final weight update equation becomes

$$w_{new} = w_{old} + \eta(t - y)x.$$

2. The bias is also updated using a similar rule.
3. The bias update equation is $b_{new} = b_{old} + \eta(t - y)$.

Intuition Behind the Rule

1. The weights increase when the predicted output is less than the target.
2. The weights decrease when the predicted output is greater than the target.
3. The perceptron gradually moves toward correct classification.
4. The rule shifts the decision boundary in the correct direction.

Example

1. Suppose input $x = 2$, target $t = 1$, predicted output $y = 0$, and learning rate $\eta = 0.5$.
2. The error becomes $e = 1 - 0 = 1$.
3. The weight change becomes $\Delta w = 0.5 \times 1 \times 2 = 1$.
4. The new weight becomes $w_{new} = w_{old} + 1$.